Aircraft Positioning in a Hangar

Technical Documentation of the Optimisation Model and Solution Methods

Baobab Soluciones

May 13, 2026

Contents

1 Problem Statement

1.1 Context

An aircraft maintenance hangar contains a fixed set of *positions* $\mathcal{P} = \{p_1, \ldots, p_P\}$ where aircraft can be parked and serviced. Each position can hold at most one aircraft at a time. A set of aircraft $\mathcal{R} = \{r_1, \ldots, r_R\}$ must be scheduled, each requiring a sequence of maintenance jobs carried out at a single position for its entire stay.

The hangar has a physical access topology: some positions can only be reached by driving through other positions. This creates *blocking arcs*: if aircraft r occupies a *front* position p while aircraft r' needs to enter or exit a *rear* position p', a ground movement is needed, generating delay and logistical cost.

1.2 Input Data

Aircraft. Each aircraft $r \in \mathcal{R}$ is characterised by:

- $e_r \geq 0$: earliest time at which r can enter the hangar.
- $T_r > 0$: target finish time (when the customer expects the aircraft back in service). Finishing after T_r incurs a delay penalty.
- A client identifier (for multi-client instances).

Jobs. Each aircraft r has an ordered sequence of maintenance jobs $\mathcal{J}^r = (j_1^r, j_2^r, \ldots)$ that must be executed *sequentially* at the assigned position:

- $d_j > 0$: processing duration of job j.
- Precedences: j must finish before j' starts whenever $(j, j') \in \mathcal{E}$. Within a single aircraft the jobs form a chain; cross-aircraft precedences are also possible.

The *total duration* of aircraft r is $D_r = \sum_{j \in \mathcal{J}^r} d_j$.

Hangar topology. The blocking structure is a directed acyclic graph (DAG):

$$\mathcal{B} \subseteq \mathcal{P} \times \mathcal{P}, \quad (p, p') \in \mathcal{B} \;\Rightarrow\; \text{an aircraft in } p \text{ blocks access to } p'.$$

A position's *blocking load* is $\ell(p) = |\{p' : p \text{ can reach } p' \text{ transitively in } \mathcal{B}\}|$.

Separation. A minimum time gap $\delta \geq 0$ must be observed between consecutive aircraft occupying the same position: if aircraft r finishes at f_r in position p, the next aircraft r' assigned to p must start no earlier than $f_r + \delta$.

1.3 Decisions

1. **Assignment**: which position $p \in \mathcal{P}$ is assigned to aircraft r. Each aircraft occupies exactly one position.
2. **Scheduling**: start time s_j for each job j, respecting precedences, earliest-start, and non-overlap within each position.

1.4 Objective Function

The objective is a weighted sum of three KPIs:

$$\min \quad w_M \cdot M + w_D \cdot \sum_{r \in \mathcal{R}} \Delta_r + w_V \cdot V \tag{1}$$

where:

Symbol	Definition	Default weight
M	Makespan: $\max_r f_r$ (time until the last aircraft leaves)	$w_M = 0.1$
Δ_r	Individual delay: $\max(0,\ f_r - T_r)$	$w_D = 1.0$
V	Total blocking movements count	$w_V = 10$

The weights above correspond to the experimental configuration used in this project. The MILP model uses higher values ($w_M = 10$, $w_D = 100$, $w_V = 1$) to avoid numerical issues; all heuristics use the same weights as the MILP when called from the experiment runner.

1.5 Blocking Movements

A *blocking event* occurs when aircraft r arrives at or departs from rear position p' while aircraft r_b is occupying a front position p such that $(p, p') \in \mathcal{B}$ and their time windows overlap.

Formally, let s_r, f_r denote the start and finish of aircraft r:

- **Entry block**: r' enters p' while $r_b \in p$: $s_{r'} \in (s_{r_b},\ f_{r_b} - \delta)$.
- **Exit block**: r' exits p' while $r_b \in p$: $f_{r'} \in (s_{r_b},\ f_{r_b} - \delta)$.

Each detected blocking event contributes 2 to V (one movement in, one out).

1.6 Feasibility

A solution is *feasible* if:

1. Every aircraft is assigned to exactly one position.
2. Jobs of an aircraft are scheduled at the assigned position, in order, without gaps within the position chain.
3. No two aircraft overlap at the same position (minimum separation δ between consecutive aircraft).
4. Each aircraft's first job starts no earlier than e_r.
5. All job precedences $(j, j') \in \mathcal{E}$ are satisfied.

2 MILP Formulation

The exact model is implemented in `models/milp_pyomo.py` as a Pyomo abstract model solved with Gurobi.

2.1 Sets

Set	Description
$\mathcal{J}$	All jobs
$\mathcal{R}$	All aircraft
$\mathcal{P}$	All positions
$\mathcal{C}$	Clients
$\mathcal{B} \subseteq \mathcal{P} \times \mathcal{P}$	Blocking arcs (p, p')
$\mathcal{E} \subseteq \mathcal{J} \times \mathcal{J}$	Job precedences (j, j')
$\mathcal{T} = \mathcal{R} \times \mathcal{R} \times \mathcal{B}$	Blocking tuples (r, r', p, p'), $r \neq r'$

2.2 Parameters

Parameter	Domain	Description
d_j	$\mathbb{R}_{\geq 0}$	Duration of job j
e_r	$\mathbb{R}_{\geq 0}$	Earliest start for aircraft r
T_r	$\mathbb{R}_{\geq 0}$	Target finish time for aircraft r
R_{jr}	$\{0, 1\}$	1 if job j belongs to aircraft r
F_{jr}	$\{0, 1\}$	1 if j is the first job of r
L_{jr}	$\{0, 1\}$	1 if j is the last job of r
δ	$\mathbb{R}_{\geq 0}$	Minimum separation between consecutive aircraft
M	$\mathbb{R}_{\geq 0}$	Big-M constant ($\max_r T_r + \sum_j d_j$)
w_M, w_D, w_V	$\mathbb{R}_{\geq 0}$	Objective weights

2.3 Decision Variables

Variable	Domain	Description
s_j	$\mathbb{R}_{\geq 0}$	Start time of job j
f_j	$\mathbb{R}_{\geq 0}$	Finish time of job j
S_r	$\mathbb{R}_{\geq 0}$	Start time of aircraft r (first job)
F_r	$\mathbb{R}_{\geq 0}$	Finish time of aircraft r (last job)
x_{rp}	$\{0, 1\}$	1 if aircraft r is assigned to position p
y_{jp}	$\{0, 1\}$	1 if job j is at position p
$z_{jj'p}$	$\{0, 1\}$	1 if job j precedes j' at position p
Δ_r	$\mathbb{R}_{\geq 0}$	Delay of aircraft r
$\hat{M}$	$\mathbb{R}_{\geq 0}$	Makespan
$\hat{V}$	$\mathbb{R}_{\geq 0}$	Total movements
Auxiliary blocking variables (per tuple $(r, r', p, p') \in \mathcal{T}$):		
$\alpha^{\text{in}}_{rr'pp'}$	$\{0, 1\}$	r' enters p' after r enters p
$\beta^{\text{in}}_{rr'pp'}$	$\{0, 1\}$	r' enters p' before r leaves p (minus δ)
$u^{\text{in}}_{rr'pp'}$	$\{0, 1\}$	Entry blocking indicator ($= x_{rp} \cdot x_{r'p'} \cdot \alpha^{\text{in}} \cdot \beta^{\text{in}}$)
$\alpha^{\text{out}}_{rr'pp'}$	$\{0, 1\}$	r' exits p' after r enters p
$\beta^{\text{out}}_{rr'pp'}$	$\{0, 1\}$	r' exits p' before r leaves p (minus δ)
$u^{\text{out}}_{rr'pp'}$	$\{0, 1\}$	Exit blocking indicator

2.4 Objective

$$\min \quad w_M \cdot \hat{M} \;+\; w_D \cdot \sum_{r \in \mathcal{R}} \Delta_r \;+\; w_V \cdot \hat{V} \tag{2}$$

2.5 Constraints

Assignment and timing.

$$\sum_{p \in \mathcal{P}} x_{rp} = 1 \qquad \forall r \in \mathcal{R} \quad \text{(each aircraft assigned to exactly one position)} \tag{3}$$

$$y_{jp} = \sum_{r \in \mathcal{R}} R_{jr} \cdot x_{rp} \qquad \forall j \in \mathcal{J},\ p \in \mathcal{P} \quad \text{(job inherits aircraft position)} \tag{4}$$

$$f_j = s_j + d_j \qquad \forall j \in \mathcal{J} \tag{5}$$

$$s_j \geq e_r \cdot F_{jr} \qquad \forall (j,r) : F_{jr} = 1 \quad \text{(earliest start)} \tag{6}$$

$$S_r = \sum_{j \in \mathcal{J}} F_{jr} \cdot s_j \qquad \forall r \in \mathcal{R} \tag{7}$$

$$F_r = \sum_{j \in \mathcal{J}} L_{jr} \cdot f_j \qquad \forall r \in \mathcal{R} \tag{8}$$

Precedences.

$$s_{j'} \geq f_j \qquad \forall (j,j') \in \mathcal{E} \tag{9}$$

Non-overlap within a position (disjunctive scheduling). Binary variable $z_{jj'p} = 1$ means job j is scheduled before job j' at position p. Using Big-M linearisation:

$$s_{j'} \geq f_j - M\big(1 - z_{jj'p} + (1 - y_{jp}) + (1 - y_{j'p})\big) \qquad \forall j \neq j',\ p \tag{10}$$

$$s_j \geq f_{j'} - M\big(z_{jj'p} + (1 - y_{jp}) + (1 - y_{j'p})\big) \qquad \forall j \neq j',\ p \tag{11}$$

$$z_{jj'p} + z_{j'jp} \geq y_{jp} + y_{j'p} - 1 \qquad \forall j < j',\ p \tag{12}$$

$$z_{jj'p} + z_{j'jp} \leq 1 \qquad \forall j < j',\ p \tag{13}$$

Aircraft-level non-overlap (minimum separation δ is embedded in the job-level constraints via the chain structure):

$$\sum_{\substack{j,j' \in \mathcal{J} \\ j \neq j'}} \big(L_{jr} \cdot F_{j'r'} \cdot z_{jj'p} + L_{j'r'} \cdot F_{jr} \cdot z_{j'jp}\big) \geq x_{rp} + x_{r'p} - 1 \qquad \forall r < r',\ p \tag{14}$$

Blocking movement indicators. For each tuple $(r, r', p, p') \in \mathcal{T}$ with $(p, p') \in \mathcal{B}$:

Entry block (r' enters p' while r is in p):

$$S_{r'} \geq S_r - M(1 - \alpha^{\text{in}}) \tag{15}$$

$$S_{r'} \leq S_r + M \cdot \alpha^{\text{in}} \tag{16}$$

$$S_{r'} \leq F_r - \delta + M(1 - \beta^{\text{in}}) \tag{17}$$

$$S_{r'} \geq F_r - M \cdot \beta^{\text{in}} - M(1 - x_{rp}) - M(1 - x_{r'p'}) \tag{18}$$

The entry blocking indicator is the linearisation of $u^{\text{in}} = x_{rp} \cdot x_{r'p'} \cdot \alpha^{\text{in}} \cdot \beta^{\text{in}}$:

$$u^{\text{in}} \le x_{rp}, \quad u^{\text{in}} \le x_{r'p'}, \quad u^{\text{in}} \le \alpha^{\text{in}}, \quad u^{\text{in}} \le \beta^{\text{in}} \tag{19}$$

$$u^{\text{in}} \ge x_{rp} + x_{r'p'} + \alpha^{\text{in}} + \beta^{\text{in}} - 3 \tag{20}$$

Exit block (r' exits p' while r is in p): Analogous constraints with $F_{r'}$ replacing $S_{r'}$, yielding u^{out}.

KPI aggregation.

$$\hat{V} = 2 \sum_{(r,r',p,p') \in \mathcal{T}} \left(u^{\text{in}}_{rr'pp'} + u^{\text{out}}_{rr'pp'} \right) \tag{21}$$

$$\Delta_r \ge \sum_{j \in \mathcal{J}} L_{jr}(f_j - T_r) \qquad \forall r \in \mathcal{R} \tag{22}$$

$$\hat{M} \ge f_j \qquad \forall j \in \mathcal{J} \tag{23}$$

2.6 Model Size

For an instance with R aircraft, J jobs, P positions, and B blocking arcs the main variable and constraint counts are:

Component	Order of magnitude
Continuous variables	$O(J + R)$
Binary variables (x, y, z)	$O(JP + J^2P)$
Blocking binary variables (α, β, u)	$O(R^2B)$
Non-overlap constraints	$O(J^2P)$
Blocking constraints	$O(R^2B)$

2.7 Solver Configuration

The model is solved with **Gurobi** via Pyomo's `SolverFactory`. Relevant options:

Option	Type	Effect
`TimeLimit`	seconds	Wall-clock budget
`MIPGap`	fraction	Stop when gap $\le$ this value (0 = optimal)
`NoRelHeurTime`	seconds	Aggressive primal heuristics before LP relaxation

3 Constructive Heuristic

3.1 Overview

The constructive heuristic (`solvers/constructive_heuristic.py`) builds solutions using a **Greedy Randomised Adaptive Search Procedure (GRASP)** combined with an optional **Iterated Local Search (ILS)** perturbation mechanism. Each iteration constructs a complete feasible solution and polishes it with a local search; the best solution found across all iterations is retained.

By default (`ils_ratio = 0.0`) the heuristic runs as pure GRASP — each iteration starts from an empty schedule. Setting `ils_ratio > 0` enables ILS: a fraction of iterations perturb the current best solution instead of building from scratch.

3.2 Algorithm

Algorithm 1 Constructive Heuristic (GRASP + ILS)

$s^* \leftarrow \emptyset$, $z^* \leftarrow \infty$
while time budget not exhausted **do**
  if $s^* \neq \emptyset$ **and** Uniform$(0,1) < \rho_{\text{ils}}$ **then**
    Remove n_{perturb} aircraft from s^*; keep the rest fixed
  else
    Start from an empty schedule
  end if
  Schedule fixed aircraft first (in earliest-start order)
  while unscheduled aircraft remain **do**
    Evaluate all (r, p) pairs over unscheduled aircraft and all positions
    Score each pair; sort by score; assign GRASP weights $w_i = (1-\alpha)^i$
    Sample one pair (r^*, p^*); commit assignment; remove r^* from pool
  end while
  if iteration $\equiv 0 \pmod{ls\_every}$ **or** current obj $< z^*$ **then**
    $s \leftarrow \text{LocalSearch}(s)$
  end if
  if $z(s) < z^*$ **then** $s^* \leftarrow s$; $z^* \leftarrow z(s)$
  end if
end while
return s^*

Joint pair selection. At each insertion step the heuristic enumerates *all* remaining (aircraft, position) pairs, not just the positions for a fixed aircraft. This means aircraft and positions are chosen simultaneously: one draw selects both who goes next and where they go. The sorted list may contain $|\mathcal{R}_{\text{free}}| \times |\mathcal{P}|$ entries; the geometric weights favour the globally cheapest pair regardless of aircraft identity.

3.3 Construction Scoring

For each candidate pair (r, p), the start time $t_s(r, p)$ is obtained from `_find_best_start`, which compares two candidate times and returns the one with lower estimated cost:

- t_{imm}: immediate start — position free time plus minimum separation δ. May create new blocking events.
- t_{nob}: no-block start — iteratively advances t_s past every blocking conflict until none remain.

The cost comparison is:

$$c(t_s,\ \text{extra\_movs}) = w_D \cdot \max(0, t_s + D_r - T_r) + w_M \cdot (t_s + D_r) + w_V \cdot \text{extra\_movs} \cdot 2$$

If $c(t_{\text{imm}}, \text{movs}_{\text{imm}}) \leq c(t_{\text{nob}}, 0)$, the immediate start is used despite the extra movements.

The GRASP score for the pair then adds a gap penalty:

$$\text{score}(r, p) = w_D \cdot \Delta_r + w_M \cdot t_f + w_V \cdot V' \cdot 2 + w_{\text{gap}} \cdot g \tag{24}$$

where V' is the total movement count after tentatively inserting r at p, and $g = \max(0,\ t_s - \texttt{pos\_free\_at}[p])$ is the idle gap created at position p (penalised only when p has already been used at least once).

The GRASP rank-based probability follows a geometric distribution:

$$P(\text{select rank } i) \ \propto\ (1-\alpha)^i, \quad i = 0, 1, 2, \ldots \tag{25}$$

$\alpha = 0$ gives **uniform random** selection (all weights equal to 1); $\alpha = 1$ gives **fully greedy** selection (only rank 0 has non-zero weight). The default value is $\alpha = 0.3$, which strongly favours top candidates while retaining stochastic diversity.

3.4 Local Search

After each construction (run every `ls_every` iterations, and always when a new best is found) the solution is polished with three operators applied in priority order until no improvement is found:

1. **Single-move**: try reassigning each aircraft to every other position; accept the first improvement.
2. **2-opt swap**: try swapping positions of every pair of aircraft with different positions; accept the first improvement.
3. **Intra-position adjacent swap**: for each position with ≥ 2 aircraft, try swapping the processing order of each pair of consecutive aircraft; accept the first improvement.

Operators are tried in the order listed; any improvement restarts from operator 1. The search is fully convergent (runs until no operator finds an improvement). Each candidate is evaluated by rebuilding the schedule in $O(R)$, giving $O(R^2 P)$ cost per pass.

3.5 ILS Perturbation

When `ils_ratio` > 0, with probability ρ_{ils} per iteration the heuristic *perturbs* the current best solution instead of building from scratch:

1. Sample $k = \min(n_{\text{perturb}},\ R - 1)$ aircraft to remove.
2. Fix the remaining $R - k$ aircraft at their current positions; schedule them in earliest-start order.
3. Re-insert the k removed aircraft using the standard GRASP loop.

3.6 Parameters

Parameter	Default	Description
`time_limit_s`	60	Wall-clock budget (seconds)
`alpha`	0.3	GRASP geometric decay
`min_separation`	10.0	Minimum gap between consecutive aircraft
`weight_makespan`	0.1	w_M
`weight_delay`	1.0	w_D
`weight_movements`	10	w_V
`weight_gap`	$= w_M$	Penalty for position idle time
`n_perturb`	4	Aircraft removed per ILS kick
`ils_ratio`	0.0	Fraction of iterations using ILS warm-start (0 = pure GRASP)
`ls_every`	1	Run local search every N iterations
`seed`	None	Random seed

3.7 Complexity

Each GRASP step evaluates $|\mathcal{R}_{\text{free}}| \times P$ pairs. Summed over all R insertion steps: $O(R^2P)$ evaluations, each requiring $O(R)$ for movement counting, giving $O(R^3P)$ per construction. Local search (first improvement, fully convergent) adds $O(R^2P)$ per pass.

4 Large Neighbourhood Search (LNS)

4.1 Overview

The LNS solver (`solvers/lns_solver.py`) iterates a **destroy–repair–accept** cycle. Starting from a good initial solution, it repeatedly removes a subset of aircraft (*destroy*), re-optimises their position assignments (*repair*), and accepts the result if it improves the incumbent.

Three repair operators are available:

- *Exhaustive repair* — enumerate all $|\mathcal{P}|^k$ combinations when $k \leq k_{\text{exact}}$.
- *GRASP repair* — $n_{\text{grasp\_repair}}$ randomised re-insertions for high iteration throughput.
- *Sub-MILP repair* — compact `gurobipy` model solved exactly for the k free aircraft; yields maximum quality per call.

The destroy strategy is selected either statically (e.g. `mixed`, `cluster`) or *adaptively* via **ALNS** (Adaptive LNS), which maintains per-strategy weights updated by a reward signal after each iteration.

4.2 Algorithm

Algorithm 2 Large Neighbourhood Search (LNS / ALNS)

if `warm_solution` provided **then**
  $s_0 \leftarrow$ `warm_solution` ▷ e.g. from TopologyHeuristic
else
  $s_0 \leftarrow \text{ConstructiveHeuristic}(t_{\text{init}})$
end if
$s^* \leftarrow \text{LocalSearch}(s_0,\ \texttt{max\_passes} = \max(5, \lfloor R/2 \rfloor))$
$s_{\text{cur}} \leftarrow s^*$
if destroy = `alns` **then**
  Initialise weights $w_d \leftarrow 1$ for each $d \in \mathcal{D}_{\text{alns}}$
end if
while time budget not exhausted **do**
  $k \sim \mathcal{U}[k_{\min}, k_{\max}]$
  $d^* \leftarrow$ selected destroy strategy (roulette if ALNS, else static)
  $\mathcal{R}_{\text{del}} \leftarrow \text{Destroy}(s_{\text{cur}}, k, d^*)$
  if $k \leq k_{\text{exact}}$ **then**
    $s \leftarrow \text{ExhaustiveRepair}(\mathcal{R}_{\text{del}})$
  else if gurobipy available **and** `repair` $\in$ {`submilp`, `auto`} **then**
    $s \leftarrow \text{SubMILPRepair}(\mathcal{R}_{\text{del}})$
    if $s = \emptyset$ **then** $s \leftarrow \text{GRASPRepair}(\mathcal{R}_{\text{del}})$
    end if
  else
    $s \leftarrow \text{GRASPRepair}(\mathcal{R}_{\text{del}})$
  end if
  $s_{\text{rb}} \leftarrow \text{Rebuild}(s)$ ▷ greedy schedule from position assignment
  if $z(s_{\text{rb}}) < z(s_{\text{cur}})$ **then** ▷ compare rebuild quality — not trial quality
    $s_{\text{cur}} \leftarrow s_{\text{rb}}$
    if $z(s_{\text{rb}}) < z(s^*)$ **then**
      $s^* \leftarrow \text{LocalSearch}(s_{\text{rb}})$ ▷ convergent polish on global best
      Update ALNS weight: global-best reward
    else
      Update ALNS weight: local-improvement reward
    end if
  else
    Update ALNS weight: no-improvement reward
  end if
  if no improvement for `restart_every` iterations **then**
    $s_{\text{cur}} \leftarrow \text{ConstructiveHeuristic}() + \text{LocalSearch}(\cdot,$ `max_passes`$)$
  end if
end while
return s^*

Accept correctness. The repair step may produce a trial solution s whose objective is computed from an optimised schedule (GRASP or Gurobi). The actual schedule stored in s_{cur} is obtained by `Rebuild`, a greedy procedure that processes aircraft in earliest-start order. Because Rebuild may yield a different (potentially worse) objective than the trial, the accept condition uses *Rebuild quality*, not trial quality. Using the trial quality for acceptance but storing the Rebuild result would silently degrade s_{cur} over many iterations.

4.3 Initialisation

Two warm-start modes are supported:

1. **External warm solution.** If `warm_solution` is passed in `params`, it is used directly as the starting point. This allows, for example, the topology heuristic to seed the LNS with a high-quality initial solution rather than spending budget on an internal constructive run.
2. **Internal constructive run.** Otherwise, the LNS spends $t_{\text{init}} = \min(10\text{ s},\ 0.15 \cdot t_{\text{limit}})$ running the Constructive Heuristic (Section 3) with $\alpha = 0.7$.

After obtaining s_0, a **bounded local search** is applied with `max_passes` $= \max(5,\ \lfloor R/2 \rfloor)$. The cap prevents the initial polish from consuming the entire time budget on large instances (e.g. $R = 30$, where an unbounded convergent search can take tens of seconds). The same bound is applied to local searches triggered by restarts.

4.4 Destroy Strategies

The destroy operator selects k aircraft to remove from the current solution. Six strategies are available:

Strategy	Description
`worst`	Remove the k aircraft with the largest individual delays Δ_r.
`most_blocking`	Score each aircraft by the number of active blocking arcs it participates in; remove the k highest-scoring aircraft.
`random`	Uniform random selection.
`mixed`	Use `most_blocking` when total delay ≈ 0; otherwise use `worst`.
`cluster`	Space-time cluster destroy (see Section 4.4.2).
`mixed_cluster`	Use `most_blocking` when total delay ≈ 0; otherwise use `cluster`.

4.4.1 Adaptive LNS (`destroy = alns`)

When `destroy = alns`, the solver maintains a weight $w_d \geq 0$ for each strategy d in $\mathcal{D}_{\text{alns}} = \{$`worst`, `most_blocking`, `cluster`$\}$. At the start of each iteration, a strategy is sampled by roulette:

$$P(\text{select } d) = \frac{w_d}{\sum_{d'} w_{d'}} \tag{26}$$

After each iteration, the weight of the selected strategy is updated with exponential smoothing:

$$w_d \leftarrow (1 - \lambda)\, w_d \ + \ \lambda\, \rho \tag{27}$$

where $\lambda = 0.85$ is the learning rate and ρ is the reward:

Event	Reward ρ
Repair improved global best s^*	3.0
Repair improved current solution only	1.0
No improvement (rebuild $\geq s_{\text{cur}}$)	0.0

All weights are initialised to 1.0. Final weights are printed at the end of each run for diagnostics.

4.4.2 Space-Time Cluster Destroy (`cluster`)

The `cluster` strategy identifies a real bottleneck — a specific place and time where several aircraft compete for the same corridor — and destroys the entire involved neighbourhood simultaneously.

Algorithm 3 Space-Time Cluster Destroy

1: **Seed selection (roulette)**: weight r as $(\Delta_r + 1) \cdot (\mathrm{bs}(r) + 1)$; sample one aircraft r^*.
2: **Spatial cluster**: $\mathcal{P}^* \leftarrow \{p_{r^*}\} \cup \{p' : (p_{r^*}, p') \in \mathcal{B}\} \cup \{p' : (p', p_{r^*}) \in \mathcal{B}\}$
3: **Temporal filter** with margin $= 0.5 \cdot \bar{D}$: $\mathcal{C} \leftarrow \{r \in \mathcal{R} : p_r \in \mathcal{P}^* \ \wedge \ s_r < t_f^* + \text{margin} \ \wedge \ f_r > t_s^* - \text{margin}\}$
4: Rank $\mathcal{C}$ by $(\Delta_r + \mathrm{bs}(r))$ descending; take top k.
5: If $|\mathcal{C}| < k$, fill remainder with highest-Δ_r aircraft outside $\mathcal{C}$.
6: **return** k selected aircraft IDs.

4.5 Repair Operators

4.5.1 Exhaustive Repair ($k \leq k_{\text{exact}}$)

All $|\mathcal{P}|^k$ position combinations are enumerated. For each combination, a complete schedule is rebuilt (`_rebuild`) and the combination minimising the objective is selected. For default settings ($k_{\text{exact}} = 4$, $|\mathcal{P}| = 5$) this evaluates at most $5^4 = 625$ combinations per iteration.

4.5.2 GRASP Repair ($k > k_{\text{exact}}$, `repair = grasp`)

$n_{\text{grasp\_repair}}$ (default 12) random trials are performed. Each trial uses the GRASP construction from Section 3 to re-insert $\mathcal{R}_{\text{free}}$ into the partial schedule defined by $\mathcal{R}_{\text{fix}}$. The best trial is kept.

4.5.3 Sub-MILP Repair ($k > k_{\text{exact}}$, `repair = submilp / auto`)

When Gurobi is available, the sub-MILP repair replaces GRASP repair with an *exact* solve of a compact subproblem. A fresh `gurobipy` model is built containing only the k free aircraft; fixed aircraft appear as time-window constraints on position occupancy intervals. A module-level `gurobipy.Env` is created once at import time to amortise the $\sim$0.5 s startup cost.

Fallback monitoring. After each run, the solver prints the sub-MILP call statistics:

```
sub-MILP calls=N feasible=M (XX%) fallback_grasp=K
```

If the feasibility rate falls below 50%, a warning is emitted recommending a reduction of `k_max` or an increase of `repair_time_s`.

4.6 Schedule Rebuild

Given a position assignment $\{(r, p_r)\}_{r \in \mathcal{R}}$, `_rebuild` computes start times by processing aircraft in earliest-start order (or an explicit order if provided — used by the intra-position adjacent-swap in local search):

$$t_s(r) = \max\bigl(e_r,\ \text{pos\_free}[p] + \delta,\ t_s^{\text{block}}\bigr)$$

4.7 Local Search

Two invocations of the local search occur:

1. **Initial and restart polish:** bounded search with `max_passes` $= \max(5, \lfloor R/2 \rfloor)$. Prevents the first polish from consuming the entire time budget on large instances.

2. **Global-best polish:** convergent search (unbounded), fired only when a repair strictly improves the global best s^*. Repairs that only improve s_{cur} are accepted without local search, preserving iteration budget.

Both invocations use single-move and 2-opt swap operators (first-improvement).

4.8 Parameters

Parameter	Default	Description
`time_limit_s`	60	Total budget (seconds)
`min_separation`	10.0	Minimum gap between aircraft
`weight_makespan`	0.1	w_M
`weight_delay`	1.0	w_D
`weight_movements`	10	w_V
`k_min`	2	Minimum destroy size
`k_max`	8	Maximum destroy size
`k_exact`	4	Threshold for exhaustive vs. GRASP/sub-MILP repair
`n_grasp_repair`	12	GRASP repair trials when $k > k_{\text{exact}}$
`destroy`	`alns`	Destroy strategy
`repair`	`auto`	Repair: `grasp` | `submilp` | `auto`
`repair_time_s`	0.2	Gurobi time budget per sub-MILP call (s)
`restart_every`	50	Restart after N non-improving iterations
`warm_solution`	None	External solution injected as warm start
`seed`	None	Random seed

4.9 Complexity and Iteration Throughput

Repair mode	Cost per iteration	Typical iters / 60 s
Exhaustive ($k \leq k_{\text{exact}}$)	$O(\lvert\mathcal{P}\rvert^k)$ rebuilds	— (always used)
GRASP ($k > k_{\text{exact}}$)	$O(n_{\text{grasp}} \cdot R \cdot \lvert\mathcal{P}\rvert)$	∼200–400
Sub-MILP ($k > k_{\text{exact}}$)	Gurobi MIP, $\leq$ `repair_time_s`	∼100–300

5 Topology-Aware Heuristic

5.1 Motivation

The constructive heuristic assigns aircraft to positions without considering the long-term effect of the hangar topology on future flexibility. When a *heavy* aircraft (large D_r) occupies a *high-load* position (one that transitively blocks many others), it holds those downstream positions inaccessible for a long time, amplifying movements and delays in a cascade.

The topology-aware heuristic (`solvers/topology_heuristic.py`) augments the GRASP score with a *topology penalty* that discourages placing urgent/heavy aircraft in high-load positions.

5.2 Topological Pre-computation

Blocking load. For each position p, the *blocking load* $\ell(p)$ is the number of positions reachable from p via the transitive closure of $\mathcal{B}$:

$$\ell(p) = \left|\{p' \in \mathcal{P} : p \xrightarrow{*} p'\}\right|$$

Positions with $\ell(p) = 0$ block nobody (safe for heavy aircraft). High-ℓ positions amplify any congestion.

The computation is a DFS per position: $O(P^2)$.

Aircraft slack. The *slack* of aircraft r measures its scheduling margin:

$$\sigma_r = \max\bigl(0,\ T_r - e_r - D_r\bigr)$$

Small σ_r means the aircraft is *urgent*: it must start soon to avoid delay.

Urgency factor.

$$u_r = \frac{1}{1 + \sigma_r/\bar{\sigma}} \quad \in\ (0,\ 1] \qquad \bar{\sigma} = \frac{1}{R}\sum_r \sigma_r$$

$u_r \to 1$ for very urgent aircraft; $u_r \to 0$ for very relaxed ones.

5.3 Algorithm

Algorithm 4 Topology-Aware Heuristic

Pre-compute $\ell(p)$ for all p; compute σ_r, u_r, $\bar{\sigma}$
$s^* \leftarrow \emptyset$, $z^* \leftarrow \infty$
while time budget not exhausted **do**
  Add Gaussian noise $\epsilon \sim \mathcal{N}(0,\ 0.005\bar{\sigma})$ to each σ_r
  Sort aircraft by $(\sigma_r + \epsilon \uparrow,\ D_r \downarrow)$ ▷ urgency order
  if LNS kick scheduled (time since last kick $\geq t_{\text{limit}}/8$) **then**
    Remove $k \in [\max(2, R/5),\ \max(3, R/3)]$ worst aircraft from s^*
    Re-schedule fixed aircraft; re-insert free with topology penalty disabled ($w_{\text{topo}} = 0$)
  else
    Construction: for each aircraft r in urgency order:
      Evaluate each position p and score:

$$\text{score}(r, p) = w_D \Delta_r + w_M t_f + w_V V' \cdot 2 + w_{\text{topo}} \cdot u_r \cdot \ell(p) \cdot w_D$$

      $\alpha_{\text{eff}} = \alpha + u_r(1 - \alpha)$; GRASP-select position
  end if
  $s \leftarrow \text{LightLocalSearch}(s,\ \max(5, R))$
  if $z(s) < z^*$ **then** $s^* \leftarrow s$; $z^* \leftarrow z(s)$
  end if
end while
return s^*

Sequential construction. Unlike the constructive heuristic (which evaluates all remaining aircraft × positions jointly), the topology heuristic processes aircraft one at a time in urgency order. For each aircraft it evaluates only the $|\mathcal{P}|$ positions and selects using biased-random GRASP. Processing in urgency order guarantees that the most constrained aircraft choose their position while the most desirable slots are still available.

5.4 Topology Penalty

The penalty added to the GRASP score when considering aircraft r at position p is:

$$\pi(r, p) = w_{\text{topo}} \cdot u_r \cdot \ell(p) \cdot w_D \tag{28}$$

Key properties:

- **Urgent aircraft** ($u_r \approx 1$): large penalty for high-load positions — they gravitate to $\ell(p) = 0$ slots.
- **Relaxed aircraft** ($u_r \approx 0$): penalty near zero — they can occupy any position without bias.
- **Zero-load positions** ($\ell(p) = 0$): penalty is always zero.

5.5 Adaptive GRASP Alpha

The GRASP randomness is adapted per aircraft:

$$\alpha_{\text{eff}}(r) = \alpha + u_r \cdot (1 - \alpha) \tag{29}$$

Recall that $\alpha = 1$ is **fully greedy** (only rank 0 selected) and $\alpha = 0$ is **uniform random** in the geometric weighting scheme $w_i = (1 - \alpha)^i$ (see Section 3). Urgent aircraft (high u_r) drive $\alpha_{\text{eff}} \to 1$, making selection nearly greedy — they pick the best available position with little randomness. Relaxed aircraft use $\alpha_{\text{eff}} \approx \alpha$, exploring more freely.

5.6 Construction Order and Noise Injection

Aircraft are processed in *urgency order*: smallest slack first, then largest duration among tied-slack aircraft. This ensures that the most constrained aircraft choose their positions while the most desirable slots are still available.

A small Gaussian noise $\epsilon \sim \mathcal{N}(0,\ 0.005\bar{\sigma})$ is added to each σ_r at the start of every iteration to diversify the ordering between iterations when aircraft have similar slack values.

5.7 LNS Perturbation Kicks

Every $\approx t_{\text{limit}}/8$ seconds (time-based, not iteration-count based) the heuristic fires a *perturbation kick* to escape topology-induced local optima:

1. Sort the current best solution by delay DESC, then finish time DESC. Take a pool of $\max(k \cdot 2,\ k+3)$ aircraft; shuffle the pool; select $k \in [\max(2, R/5),\ \max(3, R/3)]$ aircraft to remove.
2. Fix the remaining $R-k$ aircraft at their current positions; schedule them in urgency order.
3. Re-insert the k removed aircraft using topology-aware GRASP with $w_{\text{topo}} = 0$ (penalty disabled), allowing the heuristic to explore assignments that the penalty would normally prevent — including patterns where heavy aircraft occupy high-load positions with precise timing so no blocking materialises.

The time-based schedule ensures kicks fire with a consistent frequency regardless of instance size.

5.8 Bounded Local Search

The local search (`_light_local_search`) applies three operators in priority order, bounded by `max_passes` $= \max(5, R)$ outer restarts. Candidate order is shuffled between passes to increase neighbourhood diversity.

1. **Single-move**: try moving each aircraft to a different position; accept the first improvement. Aircraft and positions are shuffled before each pass.
2. **2-opt swap**: try swapping positions of every pair of aircraft with different positions; accept the first improvement. Pairs are shuffled before each pass.
3. **Intra-position adjacent swap**: for each position with ≥ 2 aircraft, try swapping every pair of consecutive aircraft in earliest-start order; accept the first improvement. This is the same operator as in Section 3.4 but uses the `_rebuild` scheduler with an explicit custom ordering.

Any improvement restarts from operator 1. The `max_passes` counter decrements only when *no* operator finds an improvement, bounding the total cost to $O(R^2 P \cdot \texttt{max\_passes})$.

5.9 Parameters

Parameter	Default	Description
`time_limit_s`	60	Total budget (seconds)
`min_separation`	10.0	Minimum gap between aircraft
`weight_makespan`	0.1	w_M
`weight_delay`	1.0	w_D
`weight_movements`	10	w_V
`weight_topology`	1.0	w_{topo}
`alpha`	0.3	Base GRASP geometric decay
`kick_interval_ratio`	0.125	Kick interval as fraction of budget (0 = disabled)
`seed`	None	Random seed

5.10 Relationship to Other Methods and Ablation Variants

Configuration	Effect
$w_{\text{topo}} = 0$	Removes topology penalty; reduces to urgency-sorted GRASP with sequential (per-aircraft) position selection.
$\alpha = 1$	Fully greedy construction (only rank 0 selected); kicks provide the only diversification.
`kick_interval_ratio` $= 0$	Disables LNS perturbation kicks entirely; solver runs as pure iterated GRASP with no large-neighbourhood escape.

These three settings define the ablation study configurations (`topology_no_penalty`, `topology_greedy`, `topology_no_kicks`) available in `run_experiments.py`.

6 Computational Experiments

This section records the results of the computational experiments run against the benchmark instance set. Each subsection corresponds to one logged experiment run; earlier runs are kept as comments for traceability.

6.1 Topology multi-start portfolio vs MILP baseline (2026-05-13)

Log file. `run_experiments_20260513_220915.log`

Context. This experiment introduces the *topology multi-start portfolio* (`topology_msN`): the 60 s budget is divided into N equal slices, each executed from a different deterministic seed (base + i), and the best solution across all starts is returned. Three portfolio sizes are compared: $N \in \{3, 6, 12\}$ (slices of 20 s, 10 s and 5 s respectively).

This run also incorporates four new intra-position local-search operators (Ops 4–7) added to `_light_local_search`: intra-position insertion, non-adjacent intra-position swap, EDD repair per block, and delay-block insertion.

Note on run artefact. Due to a configuration bug, `topology_ms3/ms6/ms12` appeared twice in the experiment list (once from `EXPERIMENTS` and once from `MULTISTART_EXPERIMENTS`), producing 7 entries instead of 4. Duplicate results are identical and both are reported here for transparency; the bug has been fixed.

Configuration. Shared parameters: $w_M = 0.1$, $w_D = 1.0$, $w_V = 10$, $t_{\text{limit}} = 60$ s, MIPGap = 0, $\alpha = 0.3$, $w_{\text{topo}} = 1$.[1]

Label	Solver	N_{starts}	Budget/start
`milp_baseline`	`MILPSolver`	—	60 s
`topology_ms3`	`TopologyHeur.`	3	20 s
`topology_ms6`	`TopologyHeur.`	6	10 s
`topology_ms12`	`TopologyHeur.`	12	5 s

Results. 34 ok / 1 failed (`milp_baseline` on `heavy-tight`, Gurobi aborted). Duplicate runs (configuration artefact) are averaged where results differ.

Table 1: `few-loose` ($R = 5$, 3 pos.). All methods reach the MILP optimal (3.12).

Solver	Obj	Makespan	Delay	Mov
`milp_baseline`	**3.12**	31.22	0.00	0
`topology_ms3`	**3.12**	31.22	0.00	0
`topology_ms6`	**3.12**	31.22	0.00	0
`topology_ms12`	**3.12**	31.22	0.00	0

[1] Weights are passed explicitly and override internal solver defaults.

Table 2: `few-tight` ($R = 5$, 2 pos.). MILP optimal = 6.39. All multi-start variants reach the optimum.

Solver	Obj	Makespan	Delay	Mov
`milp_baseline`	**6.39**	63.88	0.00	0
`topology_ms3`	**6.39**	63.88	0.00	0
`topology_ms6`	**6.39**	63.88	0.00	0
`topology_ms12`	**6.39**	63.88	0.00	0

Table 3: `many-tight` ($R = 10$, 5 pos.). MILP optimal = 4.79. Multi-start topology regresses vs single-start (6.80): shorter per-start budgets prevent the heavier LS from converging.

Solver	Obj	Makespan	Delay	Mov
`milp_baseline`	**4.79**	47.90	0.00	0
`topology_ms3`	16.93	62.59	10.67	0
`topology_ms6`	24.03	73.37	16.69	0
`topology_ms12`	24.03	73.37	16.69	0
Ref. (prev. exp.): single-start topology = 6.80				

Table 4: `many-medium` ($R = 20$, 5 pos.). MILP time-limited = 145.40. Best multi-start result: ms12 = 231.36 (run 2). Typical single-start topology ≈ 233 (multi-seed study).

Solver	Obj	Makespan	Delay	Mov
`milp_baseline`	145.40[†]	96.21	135.78	0
`topology_ms3`	246.97 / 253.81	—	—	0
`topology_ms6`	249.68 / 242.90	—	—	0
`topology_ms12`	*243.70 / 231.36*	107.91	220.56	0

[†]Time-limited feasible solution, not provably optimal. Two values = two runs (configuration artefact).

Table 5: `heavy-tight` ($R = 30$, 4 pos.). MILP aborted. Best result: ms6 run 2 = 202.92. Reference: single-start seed=None = 228.38; best of 12 seeds = 195.30.

Solver	Obj	Makespan	Delay	Mov
`milp_baseline`		*aborted*		
`topology_ms3`	254.10 / 272.20	—	—	4/0
`topology_ms6`	290.61 / ***202.92***	238.82	179.04	0
`topology_ms12`	252.15 / 272.85	—	—	0
Ref.: single-start seed=None = 228.38; best-of-12-seeds = 195.30				

Analysis.

- **Multi-start hurts small instances ($R \leq 10$).** On `many-tight` ($R = 10$), all multi-start variants regress sharply versus the previous single-start result (ms3: 16.93, ms6/ms12: 24.03 vs 6.80). The root cause is the interaction between the new heavier LS operators (Ops 4–7) and short per-start budgets: with 10 s per start (ms6) or 5 s (ms12), the solver

completes only 1–2 GRASP iterations, too few for the LS to reach a good local optimum. For $R \leq 10$, a single long start is strictly preferable.

- **Multi-start is competitive for large instances ($R = 30$).** On `heavy-tight`, ms6 run 2 achieved 202.92 — better than the previous single-start reference (228.38) and close to the best-of-12 independent seeds (195.30). This confirms that seed diversity is valuable at $R = 30$ even with shorter per-start budgets, because the main bottleneck is finding the right initial construction, not LS depth.

- **Multi-start shows high run-to-run variance at $R = 30$.** The two duplicate runs for ms6 produced 290.61 and 202.92 — a spread of 88 units — despite identical parameters. At $R = 30$ a single multi-start run is still insufficient for reliable conclusions; the best-of-K runs estimate should be used.

- **New LS operators need budget calibration.** The Ops 4–7 operators (intra-position insertion, non-adjacent swap, EDD repair, delay-block insertion) increase cost per LS pass, reducing iteration throughput. This is beneficial when the budget per start is long enough ($\geq 20\,\text{s}$) but counterproductive on short slices. A budget-adaptive N_{starts} strategy — larger N for small instances, smaller N for large — is the recommended next step.

- **Key conclusions.**

  1. For $R \leq 10$: use single-start topology (60 s). Multi-start degrades performance when per-start budget $< 20\,\text{s}$.
  2. For $R = 30$: ms3 ($3 \times 20\,\text{s}$) is a reasonable default; ms6 can yield better results but with higher variance.
  3. The new LS operators require at least $\sim$20 s per start to be effective; this constrains the maximum useful N.
  4. A duplicate experiment configuration bug has been fixed; future runs will not repeat this artefact.